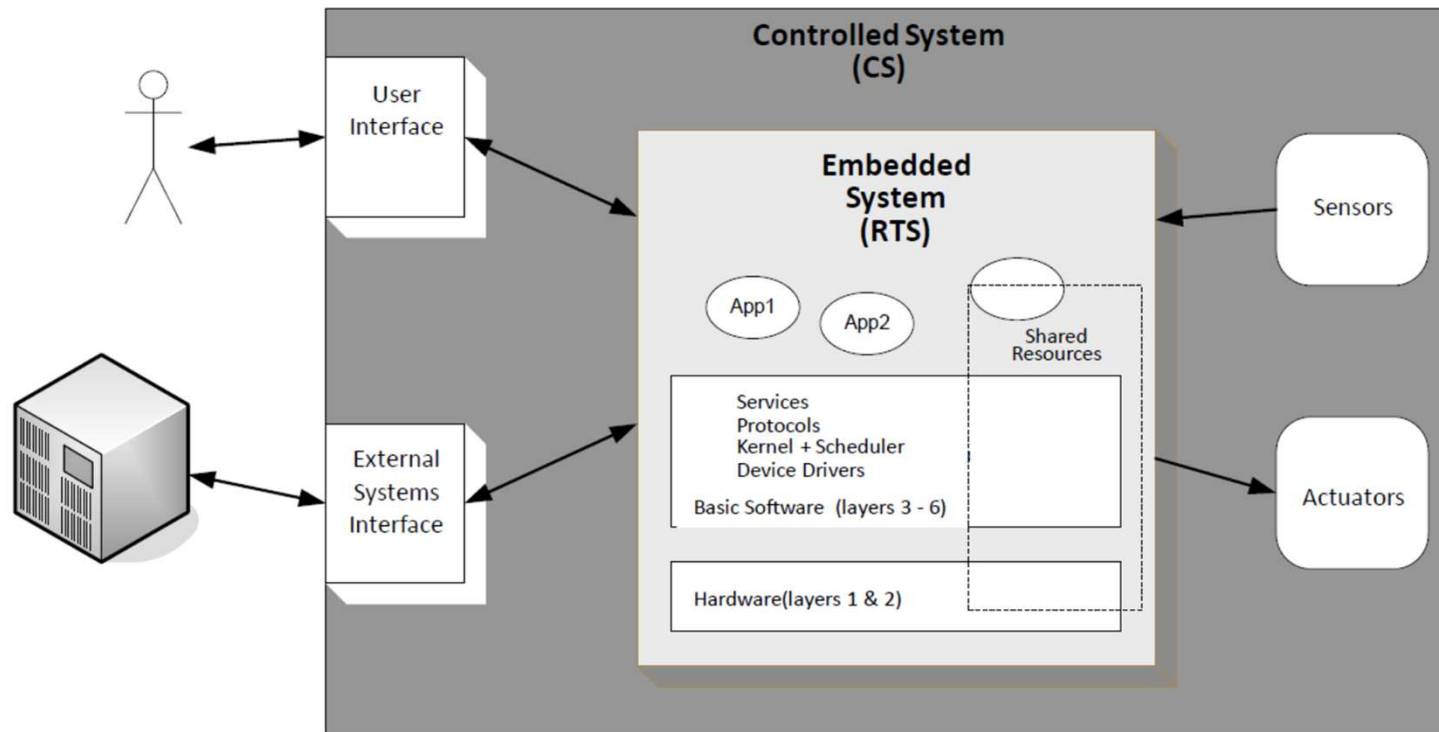


GENERIC STRUCTURE OF AN EMBEDDED SYSTEM



Renaux, D. and Stadzisz, P. , Software Embarcado. In: SBC – Escola Regional de Informática (ERI), 2007

EMBEDDED SYSTEMS ARCHITECTURE - GENERIC MODEL

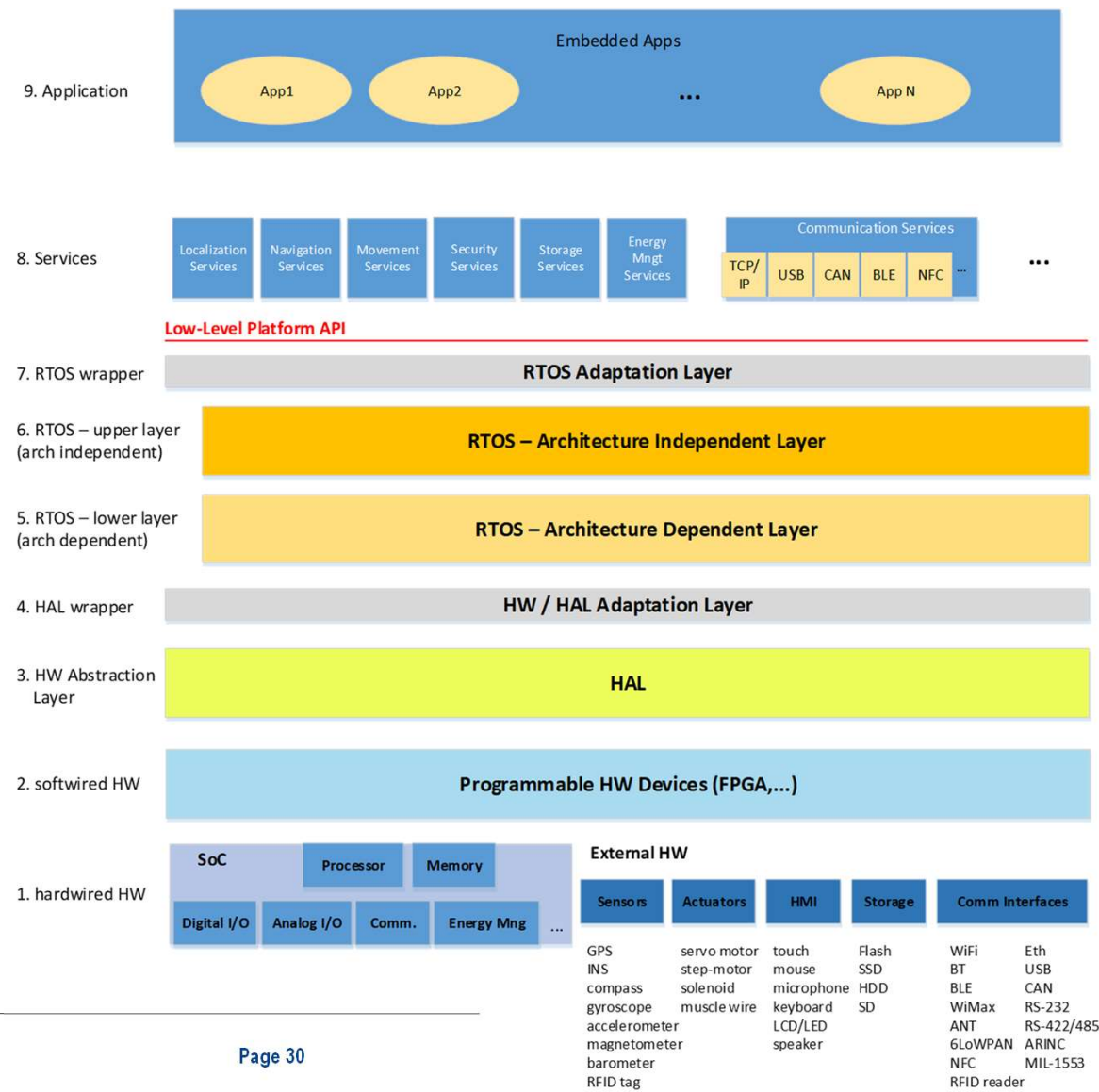
One of the important characteristics of Embedded Systems (ES) is its diversity. Hence, a truly generic model for an ES does not exist.

The model presented here attempts to represent a large set of the existing ES. Hence, it is adequate to understand Embedded Systems concepts.

suggested reading: (see IEEEXplore.ieee.org)

D. Renaux, F. Pottker, "[Applicability of the CMSIS-RTOS Standard to the Internet of Things](#)", *10th Workshop on Software Technologies for Future Embedded and Ubiquitous Systems - SEUS/ISORC 2014*, June 2014.

EMBEDDED SYSTEMS ARCHITECTURE GENERIC MODEL



source: Authors

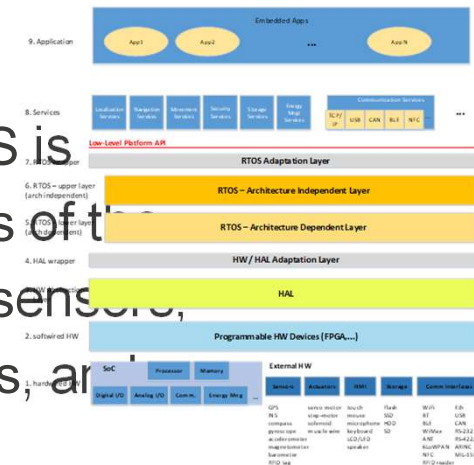
1. HARDWIRED HW

This level is composed of hardware devices (MCU, Memory, I/O, connectors) whose connection is determined by the copper traces on a PCB, hence, not changeable after fabrication.



Skywire Renesas Synergy PMOD Kit

Currently, a significant portion of the HW functionality of an ES is integrated into a SoC (System on Chip). External HW consists of the remaining HW not in the SoC and comprises, among others, sensors, actuators, Human-Machine Interface devices, Storage devices, and a large variety of communication interfaces.

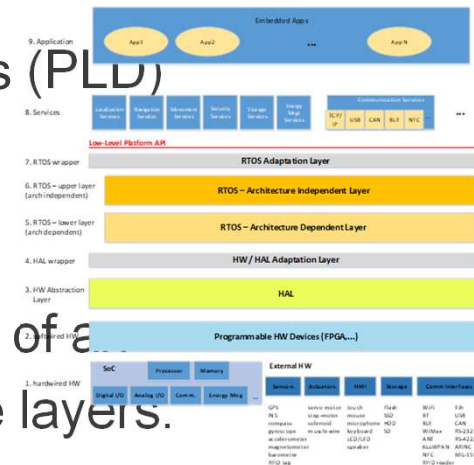


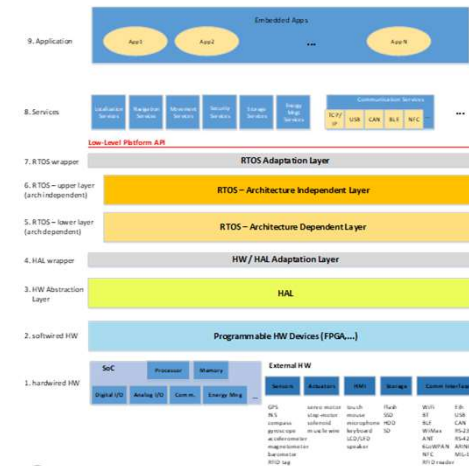
2. SOFT WIRED HW

In contrast to Level 1, the soft wired HW level consists of components whose connection is programmable, hence, can be modified at any time.

Currently, the most common programmable devices are FPGAs, however, a variety of programmable logic devices (PLD) are available: FPGA, CPLD, GAL, PAL, PLA, and even ROM.

The two hardware layers (1 and 2) compose the physical part of an embedded system. The remaining layers (3 to 9) are software layers.



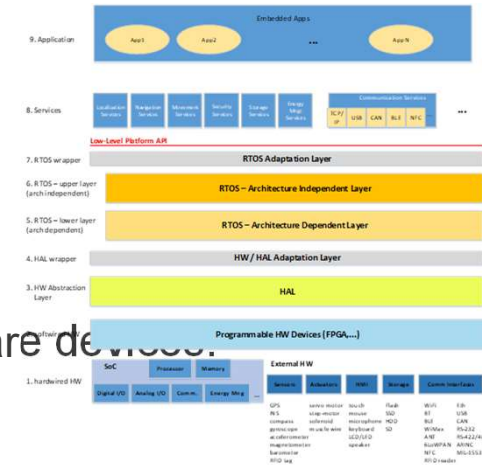


3. HARDWARE ABSTRACTION LAYER

The HAL is comprised of a set of functions that directly access the hardware devices. These functions are also called device drivers.

A well-designed HAL provides to the upper levels a standardized interface, providing an easy interchange of devices. For instance, if all communication devices have the same API then replacing a comm. interface (such as SPI) for another (such as I2C) is straightforward. The Renesas SSP (Synergy Software Package) is an example of such.

Developing a device driver for the HAL requires expertise in both hardware and software. Such a development is a complex and time-consuming task typically performed in C and sometimes mixing with assembly language.

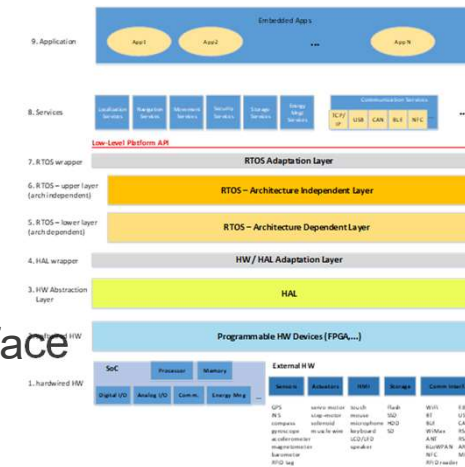


4. ADAPTATION LAYER

An Adaptation Layer (or Wrapper) is a means of providing a common interface for different device drivers.

If a HAL is not carefully designed, or if device drivers from different vendors are integrated in the same solution, then the software interface to the upper level may not be regular, meaning that different devices have access functions with different signatures. Such a scenario poses severe difficulties for portability and changes.

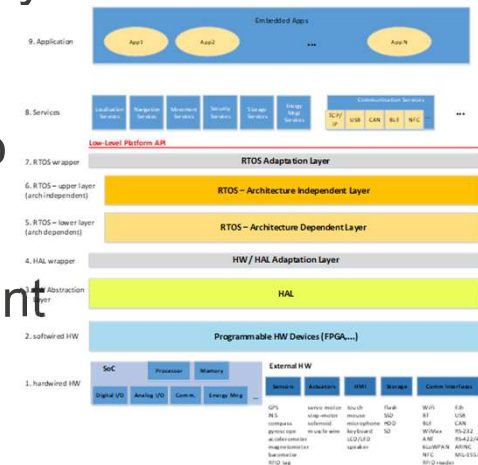
The adaptation layer is a simple translation layer aiming at converting the non-standard interface to a standardized one. Such a translation can often be done at compile time, hence, not imposing any runtime penalty.



5. and 6. RTOS

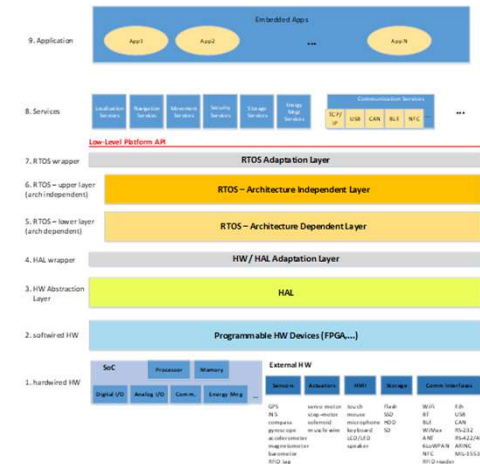
Embedded Operating Systems have significant differences to O.S. used in desktop computers, including a very small memory footprint and a reduced amount of functionality. Most embedded O.S. have to provide support for real-time systems, hence, they are termed RTOS (Real-Time Operating System).

The implementation of a well designed RTOS has at least two layers, so that different software modules implement the architectural dependent code and the architectural independent code. This approach improves modularity and improves the portability to other architectures.



7. RTOS ADAPTATION LAYER

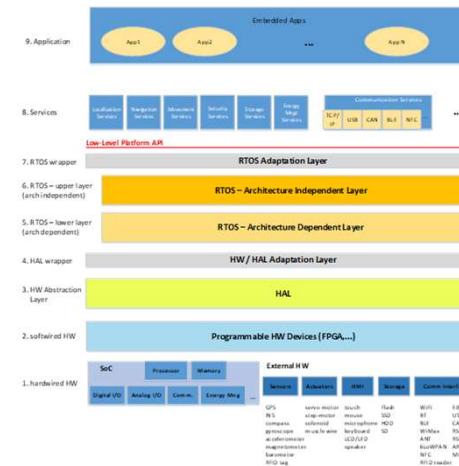
This is a wrapper that translates the API of a given RTOS to a standard API, such as CMSIS-RTOS. Once a standard API is provided to the upper levels, all of the software in the Services and Application layers can be reused in different platforms without rewriting the calls to the RTOS.



8. SERVICES

To cope with the large amount of functionality implemented by software in current Embedded Systems, code reuse is almost mandatory. Thus, off-the-shelf software components are integrated to form the final solution. Typically, these components come in the form of libraries that are linked at compile time.

A large variety of software components is available providing functionality for: TCP/IP stacks, USB stacks, communication protocols, georeferencing, navigation, security, storage, among many others.



9. APPLICATION LAYER

The top layer of the model is the Application Layer. This is the software layer that implements the specific functionality of each embedded system.

In this layer, several concurrent tasks cooperate to provide the required functionality. Concurrent programming is the most common approach to cope with the software complexity of current Embedded Systems.

The RTOS provides the management of the concurrency, among many other services.

